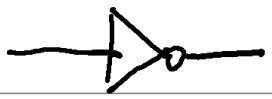
Verilog HDL

Edited by : Rium

第一讲（代码设计与仿真）

```
//反相器
// design.
module inv(A, Y)
    input  A;
    output Y;
    assign Y = ~A;
endmodule
```



// 仿真 sim

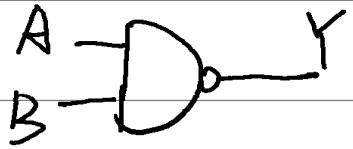
```
`timescale 1 ns / 10 ps        // 时间规模
module inv_tb;
 reg  aa ;     // 输入用 reg
 wire yy ;     // 输出用 wire
inv inv( .A(aa), .Y(yy) );

 initial begin     // 测试 aa，观察 yy.
          aa <= 0 ;
# 10     aa <= 1;  // #10 为过 10 个单位时间
# 10     aa <= 0;
# 10     aa <= 1;
# 10     $stop          // 停止
```

```
// 8位反相
module inv(A, Y);
    input [7:0]  A;
    output [7:0]  Y;
    assign       Y = ~A;
endmodule
```

//与非门

```
module nand_gate (A, B, Y);
    input  A;
    input  B;
    output Y;
    assign Y = ~(A & B);
endmodule
```



//其它逻辑门

与 A&B　　与非 ~(A&B)

或 A|B　　或非 ~(A|B)

异或 A^B　同或 A^~B 或 ~(A^B)

```
//4位与非门
`timescale 1ns/10ps
module nand_gate_4bits(A,B,Y);
    input [3:0] A;
    input [3:0] B;
    output [3:0] Y;
    assign Y = ~(A&B);
endmodule
```

```
// 4位与非门仿真
module sim;
    reg[3:0] aa, bb;
    wire[3:0] yy;
nand_gate_4bits nand_gate_4bits
(.A(aa), .B(bb), .Y(yy));
 initial begin
    aa <= 4'b0000; bb<=4'b1111;
    // 4代表位数，b是二进制
 #10 aa <= 4'b0011; bb<=4'b0110;
 #10  $stop;
 end
 endmodule
```

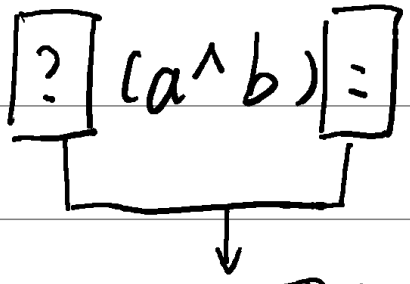
第二讲（组合逻辑）

//二选一逻辑

/* sel为0 y = a & b

sel为1 y = a ^ b */

```
module fn_sw (a, b, sel, y);
    input a; input b; input sel;
    output y;
    assign y sel ? (a^b) : (a&b)
endmodule
```



和C++逻辑相同.

// always 语句二选一

```
y = sel ? (a^b) : (a&b);
```

↓替换为

```
reg y;  //不能漏
always @ (a or b or sel)
        // a, b, sel 成为敏感变量
begin
  if(sel == 1) begin
    y <= a^b;
  end
  else begin
    y <= a&b;
  end
end
```

```
// always 4选1
  reg y;
  always @(a or b or sel) begin
      case (sel)
      2'b00: //.....
      2'b01: //.....
      2'b10: //.....
      2'b11: //.....
      endcase
  end
endmodule
```

```
// 4选1仿真
module sim;
  reg [3:0] absel;
  wire      y;

  fn_sw_4 fn_sw_4
  (.a(absel[0]), .b(absel[1]),
   .sel(absel[3:2]), .y(y));
  //[3:2]为最后两位

  initial begin
            absel<= 0;
    #200 $stop
  end
```

(后接)

(前继)

always #10 absel <= absel + 1;

//每隔10个单位时间，absel ++

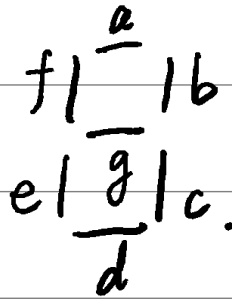
第三讲(补码转化，七段管)

原码 → 正 → 原码 = 补码
原码 → 负 → 符号位不变，幅度位取反，+1.

```
module comp_conv( a, a_comp);
    input [7:0] a;
    output [7:0] a_comp;
    wire [6:0] b;          //y = {a[7],
    wire [7:0] y;          //    ~a[6:0]+1};
    assign b = ~a[6:0];
    assign y[6:0] = b + 1;
    assign y[7] = a[7];
    assign a_comp = a[7] ? y : a;
endmodule
```

仿真

```
`timescale
module sim;
reg [7:0]  a_in;
wire [7:0]  y_out;
comp_conv comp_conv (
        .a(a_in),
        .a_comp(y_out)
        );
initial begin
        a_in <= 0 ;
        #3000 $stop;
end
alway #10  a_in<= a_in+1;
endmodule
```



七段码译码.

```
module seg_dec (num, a_g)
    input [3:0] num; output [6:0] a_g;
    reg [6:0] a_g;
    always @ (num) begin
      case(num)
      4'b0: a_g <= 7'b111_1110;
      //.....
      4'b101: a_g <= 7'b111_1011;
      default: a_g <= 7'b000_0000;
    endcase
endmodule
```

第四讲(时序逻辑代码设计与仿真)

```
// D触发器（复位输出0，未复位输出d）
module D_FF(
    input clk, input rst_n,
    input d, output q
);
    reg q;
                //上升沿有效          //下降沿有效
    always@(posedge clk or negedge
            rst_n) begin
        if(!rst_n) q <= 1'b0;
        else q<=d;
    end
endmodule.
```

```
// JK触发器 (同0保持，同1取反，不同取j)
module JK_EF(
    input clk, input rst_n,
    input j, input k, output q
);
    wire clk, rst_n, j, k;
    reg q;
    always @(posedge clk or negedge
                rst_n) begin
        if(!rst_n) q<=1'b0;
        else begin
            case({j,k})
            2'b00: q<=q;
            2'b01: q<=j;
            2'b10; q<=j;          (后接)
```

```
            2'b11: q <= ~q;
        end
    end
endmodule

//T触发器（行为描述）
module T_EF(
    input clk, input rst_n,
    input t , output q
);
    reg q;

    always @(posedge clk or negedge
                rst_n) begin
```

（后接）

```
        if(!rst_n) q <= 1'b0;
        else if(t == 1'b0) q <= q;
             else          q <= ~q;
    end
endmodule
// T触发器结构描述.(D触发器构造).
module TFF(input wire clk,
        input wire rst_n, input wire T,
        output Q);
wire d;
wire q;

XOR2X1 XOR_INST(.A(T), .B(q),
            .Y(d));
```

```
DFFRX1 DFF_INST (
    .D(d),
    .CK(clk),
    .RN(rst_n),
    .Q(q),
    .QN()
);
 assign Q = q;
endmodule
```

```
// 四级伪随机码发生器.
module m_gen(clk, res, y);
    input  clk;
    input  res;
    output  y;
    reg [3:0] d; // 4位触发器.
    assign   y = d[0];
    always @(posedge clk or
                negedge res).
    if (~res) d <= 4'b1111;
    else begin
            d[2:0] <= d[3:1];
            d[3] <= d[3] + d[0];
    end
endmodule.
```

```
// 秒计数器.
module  s_counter(
        input  wire  clk,
        input  wire  rst_n,
        output [3:0]  s_num
);

    parameter  f = 24000000 // 24MHz
    reg  [24:0]    cnt ; // 分频计数.
    reg            s_pulse; // 秒脉冲尖
    reg [3:0]   s_num ;


    always @ (posedge clk or.
                negedge rst_n) begin.
```

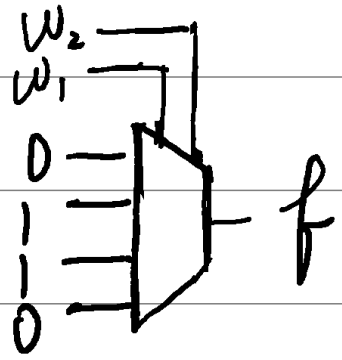
```
if (!rst_n) begin
    cnt <= 0;
    s_pulse <= 0;
end else begin
        if (cnt = f-1) begin
            cnt <= 0;
            s_pulse <= 1;
        end
        else cnt <= cnt + 1;
end
```

```
    if (s_pulse) begin
        if (s_num == 9)  s_num <= 0;
        else begin
            s_num <= s_num + 1;
        end
    end
end
endmodule
```

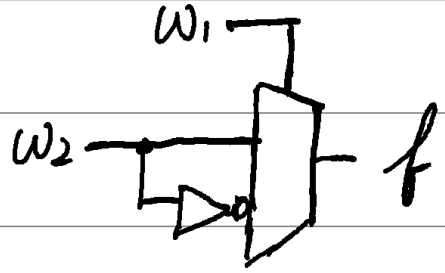
always @ (s_pulse) begin .

组合逻辑

§ 选择器



w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0



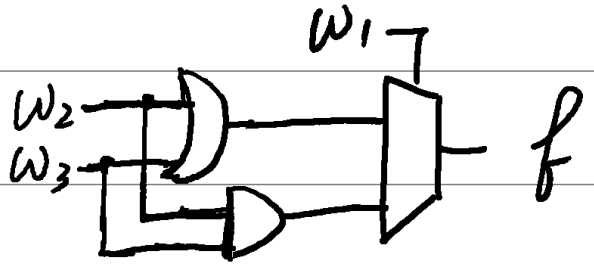
§ 香农展开

$$f(w_1, w_2, \cdots, w_n) =$$
$$\bar{w_1} f(0, w_2, \cdots, w_n) +$$
$$w_1 f(1, w_2, \cdots, w_n).$$

e.g. $f(w_1, w_2, w_3) = w_1 w_2 + w_1 w_3 + w_2 w_3$

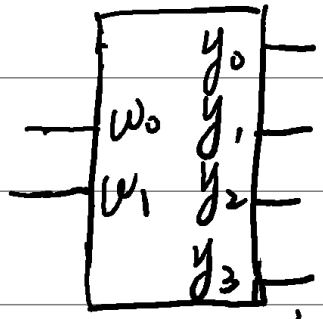
$$= \bar{w_1}(0 w_2 + 0 w_3 + w_2 w_3) +$$
$$w_1 (w_2 + w_3 + w_2 w_3).$$

$f(w_1, w_2, w_3) = \bar{w}_1 w_2 w_3 + w_1(w_2 + w_3)$

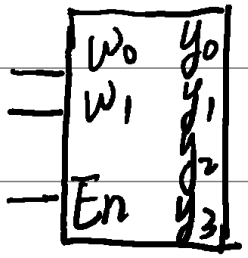


§二进制译码器

w_0	w_1	y_0	y_1	y_2	y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



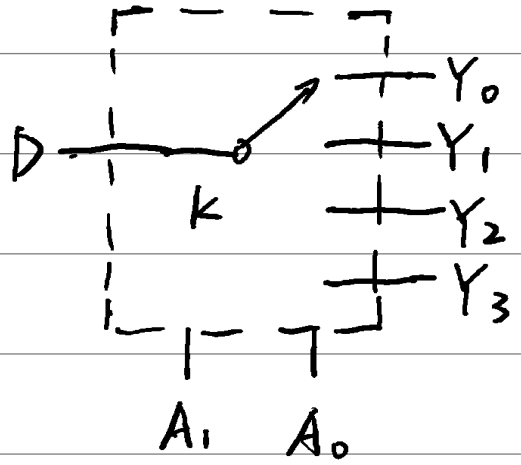
独热编码



En = 1 时，
译码器才工作.

* 一个n位二进制码中有且只有1位取值为1，称为独热码。

§ 数据分配器 DEMUX

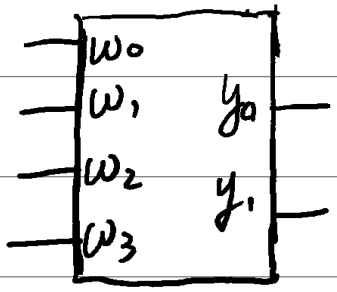


A_1 A_0	Y_0	Y_1	Y_2	Y_3
0 0	D	0	0	0
0 1	0	D	0	0
1 0	0	0	D	0
1 1	0	0	0	D

功能类似多位开关，

将D送到选择输入指定通道上。

§ 编码



w_0	w_1	w_2	w_3	y_1	y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

优先编码器

当一个高优先级输入插入时，低优先级输入被忽略.

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	—	—	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

$i_0 = \overline{w_1}\,\overline{w_2}\,\overline{w_3}\,w_0$　　$i_1 = \overline{w_2}\,\overline{w_3}\,w_1$

$i_2 = \overline{w_3}\,w_2$　　$i_3 = w_3$

$y_0 = i_1 + i_3$. $z = i_0 + i_1 + i_2 + i_3$

$y_1 = i_2 + i_3$

§二选一选择器.

```
module mux_2 (
    input wire w0,
    input wire w1,
    input wire s,
    output wire f
);.
    assign f = s ? w1 : w0;.
endmodule
  /* 另一种描述                    \
    reg f;
    always @(w0, w1, s)
```

```
    f = s ? w1 : w0;
endmodule
  \ 可用 if-else 语句.                    */

// 用5个 mux4 to 1 构造 mux16 to 1.
module mux16to1 (
    input  wire [15:0]   W;
    input  wire [3 :0]   S;
    output  f.
);
    wire [3:0] tmp; .// 不能为 reg

    mux4to1  Mux1 (W[3:0], S[1:0], tmp[0]);
    mux4to1  Mux2 (W[7:4], S[1:0], tmp[1]);
    mux4to1  Mux3 (W[11:8], S[1:0], tmp[2]);
```

```
    mux4to1 Mux4 (W[15:12], S[1:0], tmp[3]);
    mux4to1 Mux5 (tmp[3:0], S[3:2], f);
endmodule.
```

§ casez/x.

casez：将z作无关项，(优先解码器).
可忽略高阻态.

casex：将x，z作无关项.

x：未知值/冲突

z：高阻态，三态门输出，断开.

?：通配，不关心.

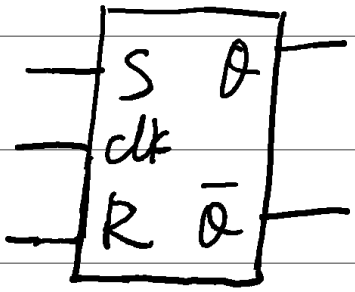
1'bx 与 1'bx 间比较用“===”，“==”结果为x.(z也一样)

```
//二位二进制解码器.(独热码)
module dec2to4(
    input wire [1:0] W,
    input En,
    output reg [3:0] Y
);
    always @(W, En) begin
        case({En, W})
            3'b100: Y = 4'b1000;
            3'b101: Y = 4'b0100;
            3'b110: Y = 4'b0010;
            3'b111: Y = 4'b0001;
            default: Y = 4'0000;
        endcase.
endmodule
```

触发器、寄存器&计数器.

§ 门控 RS 锁存.

S	R	Q(t+1).
0	0	Q(t).
0	1	0
1	0	1
1	1	x



R·S = 0 · (约束条件).

* 一般为电平敏感.

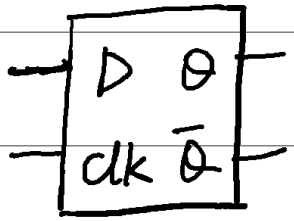
$Q^{(n+1)} = S + \bar{R}Q^{(n)}$ (NOR, 高电平)

$Q^{(n+1)} = \bar{S} + RQ^{(n)}$ (NAND, 低电平)

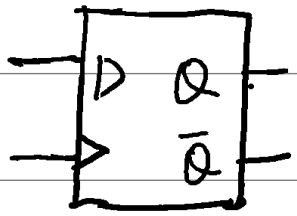
§ D 锁存.

$Q^{(n+1)} = D$.

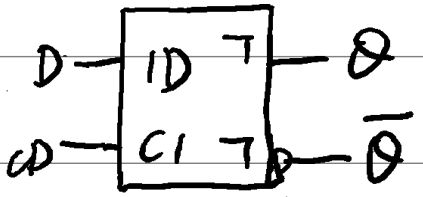
D	$Q^{(n+1)}$
0	0
1	1



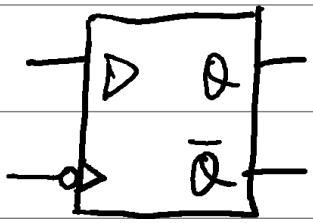
5 主从D触发器



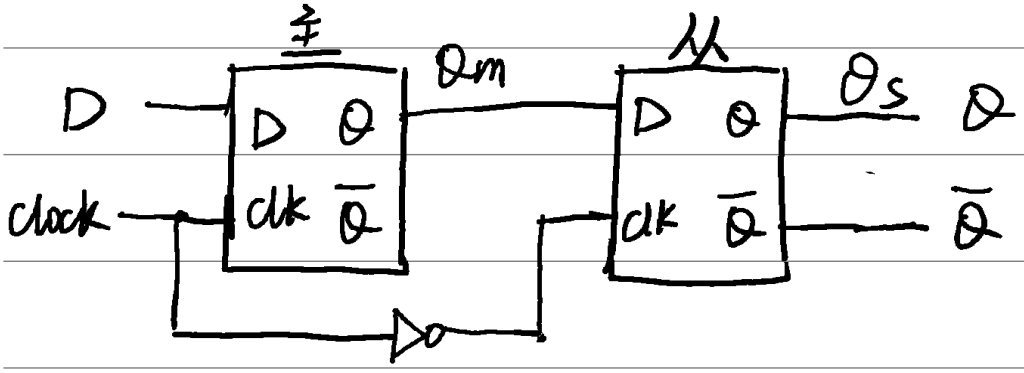
边沿触发

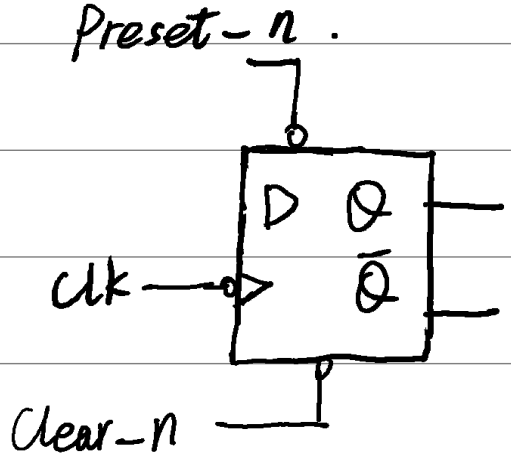


(国际符号).



负边沿触发.



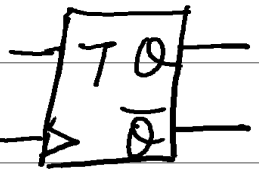


带预置和清零

§ T触发器.

$$Q^{(n+1)} = T \oplus Q^{(n)}$$

T	Q(t+1)
0	Q(t)
1	$\overline{Q}(t)$.



§ JK触发器.

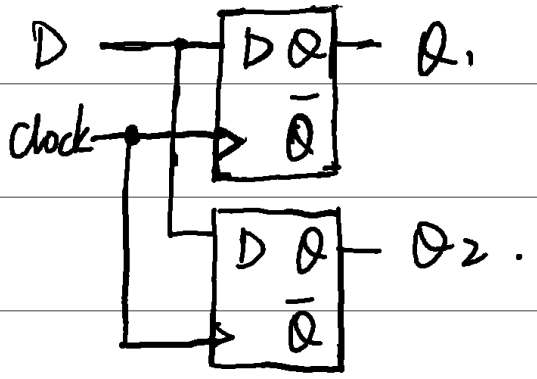
$$Q^{(n+1)} = J\overline{Q}^{(n)} + \overline{K}Q^{(n)}$$

J	K	Q(t+1)
0	0	Q(t)
0	1	0
1	0	1
1	1	$\overline{Q}(t)$

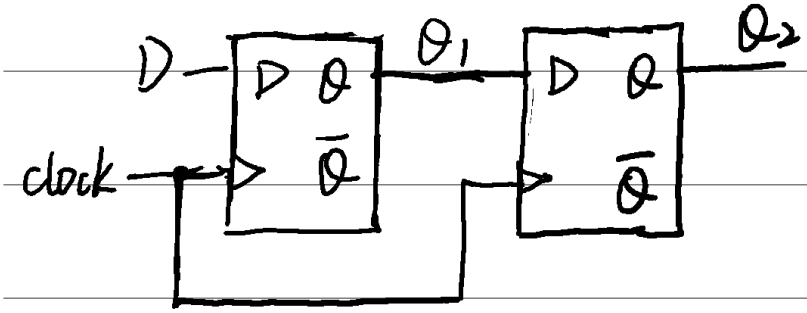
§ 阻塞与非阻塞赋值.

```
module exp (
    input wire D,
    input wire clock,
```

```
    output reg Q1,
    output reg Q2
);
    always @(posedge clock) begin
        Q1 = D;
        Q2 = Q1;    // "=" 阻塞赋值.
    end
endmodule
```



```
always @(posedge clock) begin
    Q1 <= D;
    Q2 <= Q1; // "<="非阻塞赋值
end
endmodule
```



$Q_1^{(n+1)} = D$

$Q_2^{(n+1)} = Q_1^{(n)}$.

只能用非阻塞赋值描述时序电路.

§ 4位移位寄存器.

```
module shift4 (
    input  wire [3:0] R,
    input  wire load,
    input  wire clk,
    input  wire write,
    output reg [3:0] Q
);
    always @ (posedge clk) begin
        if(load) Q <= R;
        else begin
            Q <= {write, Q[3:1]};
        end.    //右移.
    end.
endmodule
```

5 递增/减计数器

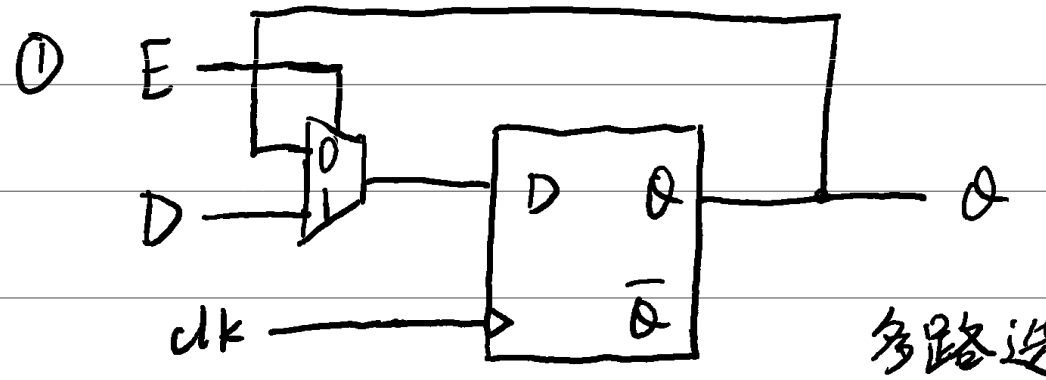
```
module up_down_count #(
    parameter n = 8
)(
    input wire [n-1:0] read,
    input wire clk,
    input wire load,
    input wire en,
    input wire mode,
    output reg [n-1:0] Q
);
    always @(posedge clk) begin
        if(load) Q <= read;
        else if(en) begin
            Q <= Q + (mode ? 1 : -1);
        end
```

```
    end.
endmodule.
  /* mode 为 1，计数 +1，  \
  \   mode 为 0，计数 -1   */
```

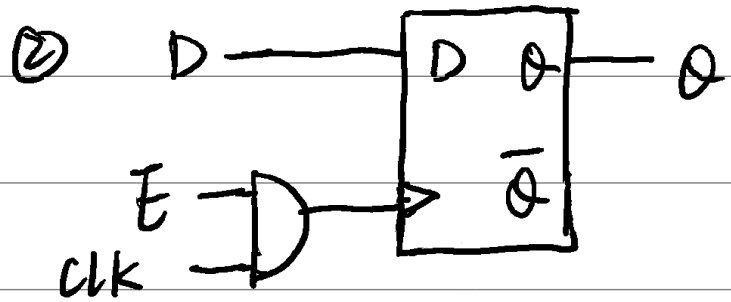
§ 带使能输入的触发器 & 寄存器

$E=0$，$Q_{n+1}=Q_n$.

$E=1$，$Q_{n+1}=D$. // 由外界写入.

① 

多路选择器.

② 

采用门控时钟.

同步时序电路.(有限状态机).

Moore 状态表.(输出只由状态决定)

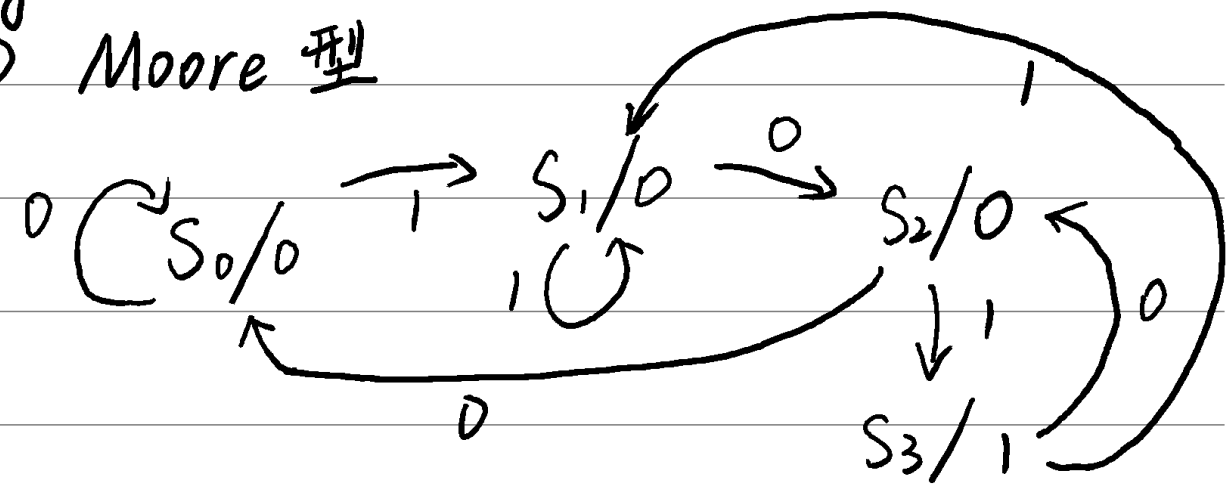
当前状态	下一状态.		输出.
	W=0	W=1	

Mealy 状态表.

当前状态	次态/输出.	
	W=0	W=1.

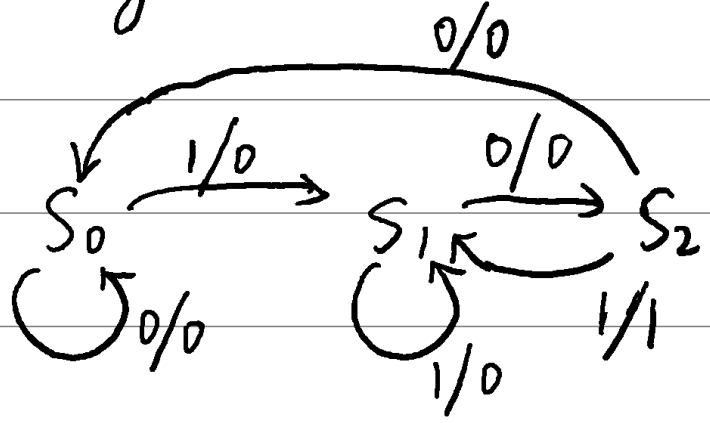
e.g. 101 序列检测.

① Moore 型



现态	次态		输出
	W = 0	W = 1	
S_0	S_0	S_1	0
S_1	S_2	S_1	0
S_2	S_3	S_0	0
S_3	S_2	S_1	1

② Mealy 型



现态	次态/车输出	
	W=0	W=1
S_0	$S_0/0$	$S_1/0$
S_1	$S_2/0$	$S_1/0$
S_2	$S_1/1$	$S_0/0$

§设计.

① 确定状态数 S_0, S_1, …, S_n.

② 画状态图 & 表.

③ 有必要时状态化简，减少 S_i 数.

④ 状态编码 S_0、S_1、S_2、S_3

↓

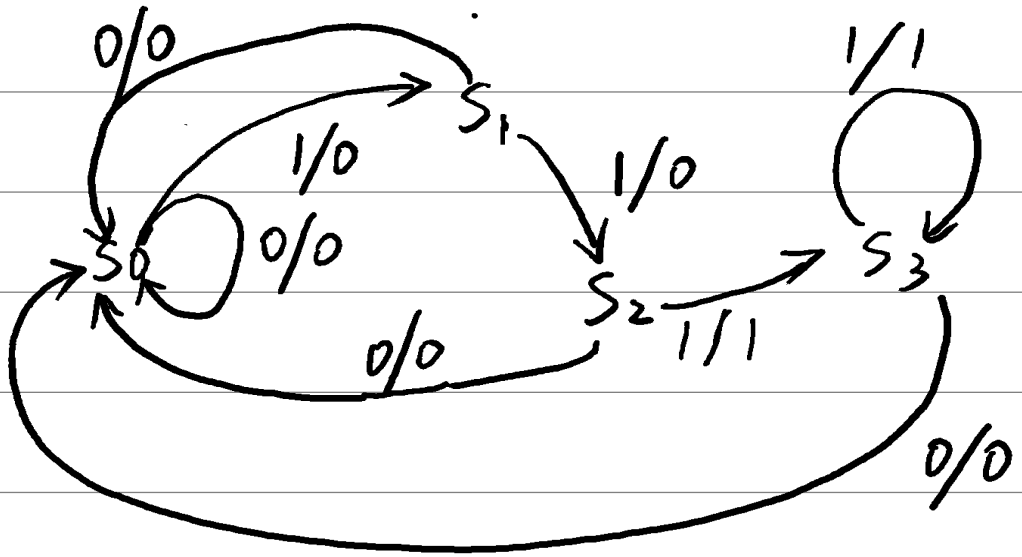
Q_1Q_0.

⑤ 根据真值表列状态方程.

⑥ 画电路图

e.g. 111序列检测

① 原始状态图/表.



现态	输入 X=0	X=1
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_0/0$	$S_3/1$
S_3	$S_0/0$	$S_3/1$

② 最简状态图/表

同原始状态图/表

③ 二进制状态表（二进制编码）

现态 $Q_1^{(n)}$ $Q_0^{(n)}$	输入	次态 $Q_1^{(n+1)}$ $Q_0^{(n+1)}$	输出
0 0	0	0 0	0
0 0	1	0 1	0
0 1	0	0 0	0
0 1	1	1 0	0
1 0	0	0 0	0
1 0	1	1 1	1
1 1	0	0 0	0
1 1	1	1 1	1

④ 输出函数 & 激励函数

$$Q_0^{(n+1)} = X\cdot(\overline{Q_1^{(n)}}\,\overline{Q_0^{(n)}} + Q_1^{(n)}\overline{Q_0^{(n)}} + Q_1^{(n)}Q_0^{(n)})$$

$$= X\cdot(Q_1^{(n)} + \overline{Q_0^{(n)}})$$

$$Q_1^{(n+1)} = \overline{Q_1^{(n)}}Q_0^{(n)}X + Q_1^{(n)}\overline{Q_0^{(n)}}X + Q_1^{(n)}Q_0^{(n)}X$$

$$= X(\overline{Q_1^{(n)}}Q_0^{(n)} + Q_1^{(n)})$$

$$= X(Q_0^{(n)} + Q_1^{(n)})$$

$$Y = X\cdot Q_1^{(n)}$$

(1) D 触发器版. (激励函数).

$$D_0 = X\cdot(\overline{Q_1^{(n)}} + Q_0^{(n)})$$

$$D_1 = X\cdot(Q_0^{(n)} + Q_1^{(n)})$$

(2) JK 触发器版. (激励函数).

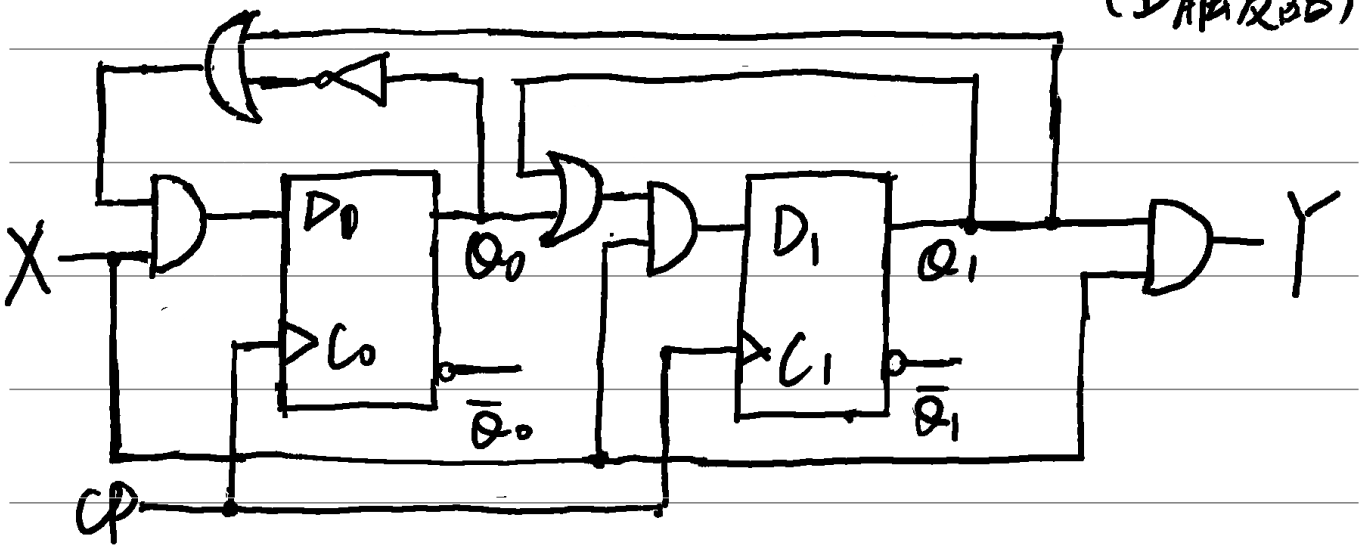
$$Q_0^{(n+1)} = XQ_1^{(n)}Q_0^{(n)} + X\overline{Q_0^{(n)}}$$

$$J_0 = X, \quad K_0 = \overline{X} + \overline{Q_1^{(n)}}$$

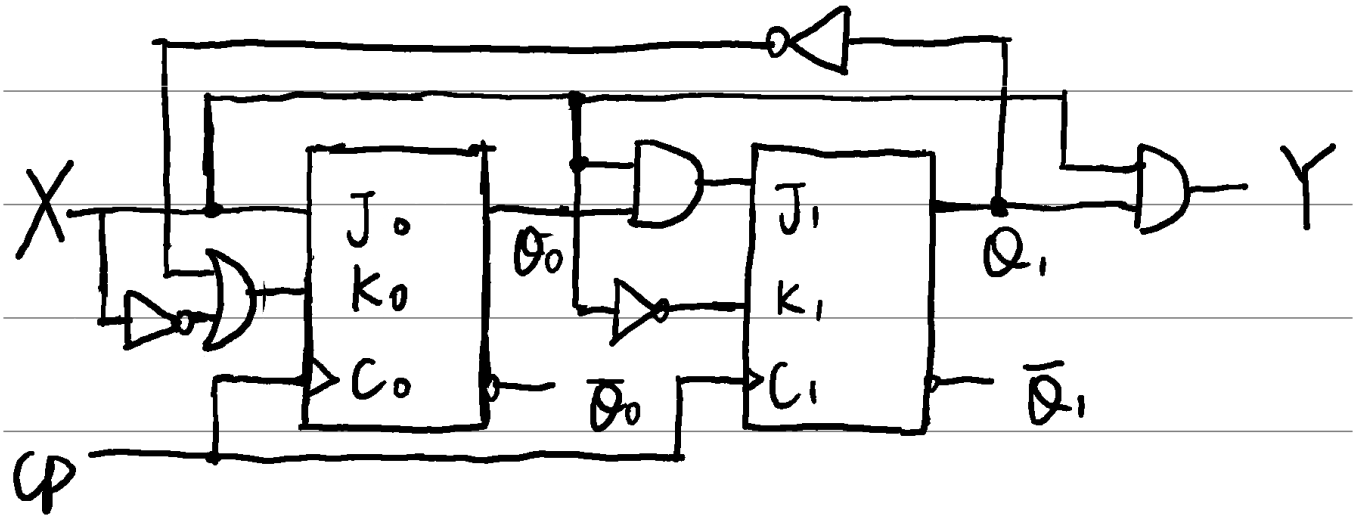
$$J_1 = XQ_0^{(n)} \quad K_1 = \overline{X}.$$

⑤ 逻辑电路图

(D触发器)



(JK触发器)



§ BCD码：4位二进制对应一位十进制

(1) 8421码

925 ⇒ $\underset{9}{|100|}$ $\underset{2}{00|0}$ $\underset{5}{0|0|}$.

(2) 2421码.

最高位的"1"代表"2".

9 ⇒ 1111 // 2+4+2+1

(3) 余三码

9 ⇒ +3=12 ⇒ 1100

2 ⇒ +3=5 ⇒ 0|0|

5 ⇒ +3=8 ⇒ 1000

§ 循环码(格雷码). 相邻只变1位

000 → 001 → 011 → 010

→ 110 → 111 → 101 → 100

→ 000

§ 奇偶校验码.

添加1位校验位使1的个数为奇/偶. (添加至首位).

§ 对偶式与相反式

① 与或互换.

② 01互换.

} 对偶

③ 变量保持不变.

下标相同的最大项和最小项互为相反.

同或和异或同时互为相反与对偶

§ 冒险与竞争.

竞争：信号传播延迟不同，导致多信号到达同一逻辑门时间不一致.

冒险：竞争引起的瞬间错误/毛刺

e.g. wire F = (A & B) | (~A & C)

// B、C为1时，F应保持为1

// A发生变化，可能会出现毛刺.

$F = A + \overline{A}$ 0冒险.

$F = A * \overline{A}$ 1冒险

#卡诺图判别法：有相切不相交的圈，即有冒险可能.

§ Timescale

`timescale 5ns/1ps

5 ns：时间单位，#1 ⇒ 5ns

1 ps：时间精度，仿真器能分辨最小精度.

§ 大小项.

① 小项

$\bar{A}\bar{B}\bar{C}$ m_0 000 → 0.

$\bar{A}B\bar{C}$ m_2 010 → 2.

② 大项.

$A+B+C$ M_0

$A+\bar{B}+C$ M_2

性质：$M_i = \overline{m_i}$ (互补关系).

M_i 与 m_i 为相反式

§ 进制转化

原码 (+3) 0000，0011

(-3) 1000，0011.

反码 (+5) 0000, 0101 正数同原码

(-5) 1111, 1010

负数对正数按位取反.

补码 (+127) 0111, 1111 正数同原码

(-127) 1000, 0001

负数：反码+1.

8位字长补码可表示 -128 ~ 127

十进制小数转二进制.

$(0.8125)_{10} \Rightarrow (0.1101)_2$

$0.8125 \times 2 = 1.625$.

$0.625 \times 2 = 1.25$.

$0.25 \times 2 = 0.5$

$0.5 \times 2 = 1.0$.

$(0.2)_{10} \Rightarrow (0.\dot{0}0\dot{1}1\dot{)}_2$

$0.2 \times 2 = 0.4$

$0.4 \times 2 = 0.8$

$0.8 \times 2 = 1.6$

$0.6 \times 2 = 1.2$

$0.2 \times 2 =$ …… 出现循环

进位 & 溢出

进位：运算结果超出位数，向前进位自然丢失，不影响结果对错.

32　　0010　0000

−25　　+1110　0111

7　　自然丢失 ① 0000　0111

溢出：进位造成结果出错.

127	0111 1111
+ 1	+ 0000 0001
128	1000 0000

$127+1\neq -128$.

§ 逻辑代数基础.

与或式.

$F=AB+A\bar{C}+BC$. 先与后或.

或与式.

$F=(A+B)\cdot(A+\bar{C})\cdot(B+C)$ 先或后与.